INSTITUTO TECNOLOGICO DE CULIACAN

Mauricio Lopez Lara
Carlos Ley Garcia
30/03/2026
Tópicos avanzados de IA
N Reynas Tarea 2 BONUS

A continuación se presentara un problema a resolver, este problema se basara en un juego virtual o de mesa el cual lleva por nombre N reinas, es parecido al ajedrez solo que en este juego consiste en acomodar piezas pero con reglas estrictas a seguir, por ejemplo tenemos un tablero de 4x4 lo que eso significa que tenemos 4 reinas y como sabemos una reina en el ajedrez la pieza de la reina es la que manda es la que ataca en línea recta por cualquier lado sea arriba, abajo, izquierda o derecha y si lo ponemos a pensar la visión de la pieza es cruzar todo el tablero, en el N reinas para poder concluir el ejercicio todas las reinas debes ponerlas todas en el tablero al mismo tiempo solo que la regla para que ganes es la de que ninguna reina puede cruzarse en la línea de ataque de otra dicho de un modo no debe colisionar, deben estar acomodadas de tal forma en que ninguna colisione, ahora que sabemos en que consiste el juego del N reinas ahora quisiera hablar del tabú, el método tabú definiéndolo de manera rápida es un método que utilizamos para poder ver múltiples soluciones e identificar cual es la mejor pero esto dependerá de que contexto le demos en este caso es el N reinas y el objetivo del N reinas es el poder encontrar la mejor solución para que ninguna reina en el tablero colisione y de manera practica uno pueda ganar el juego, dicho esto empezaremos con una breve descripción del problema pero más detallada, después representaremos los parámetros y las soluciones, se dará un propuesta del algoritmo, su implementación en lenguaje Python, para terminar se hablara sobre sus métricas de tiempo y cantidad de movimientos junto con sus reglas.

1. Problema a resolver

Para este problema colocaremos 8 reinas lo cual seria un tablero de 8x8 desde el principio y cada una de las reinas estará en cada columna en posiciones aleatorias, cada una de ellas se estarán atacando entre si, el objetivo principal de la función es contar cuantas colisiones hay en el tablero actualmente y lo que queremos es minimizarlo hasta que el numero llegue a 0 esto significa que el problema se detendrá cuando ya haya encontrado la solución perfecta, en cada turno tomaremos una reina y la moveremos a otra fila dentro de su misma columna esto se hace porque si ponemos esta restricción de movimiento solo se podrá moverse de arriba abajo y en las reglas del N reinas es que ninguna choque en fila, columna o diagonal entonces tomando esto en cuenta estamos garantizando de manera matemática que la regla de las columnas es improbable que se rompa, para la regla de Tabú para evitar el clásico problema que el programa se quede atrapado en un ciclo infinito o mejor dicho en un ciclo donde la misma reina se mueva de arriba a bajo en el mismo tablero entonces se guardaran los últimos tableros visitados en una lista tabú y el programa como regla estricta tiene prohibido elegir que este en esa lista ya mencionada y por que se hace esto pues mayormente por los ciclos infinitos ya mencionado y también para obligar al programa a salir de su zona y esto es la

famosa lista tabú y esta lista lo que es, es una memoria para el algoritmo que estamos usando barra creando.

2. Representaciones de sus parámetros y sus soluciones

Para estas representaciones haremos uso de matemáticas simples:

El parámetro es encontrar una configuración de tamaño N=8 para que el numero de choques sea igual a 0 o como ya se explico que sea la mejor solución.

El espacio de soluciones se representará en el estado del tablero como X como una sencilla lista de 8 números enteros y no decimales porque si son decimales los cálculos se volverían más probabilidad.

El índice de la lista estará representará la columna el cual en este caso seria del 0 al 7

El valor en la posición representará la fila que al igual que el índice será de 0 al 7

Por dar un ejemplo si tenemos que la solución es [2,5,1,2,1, 3, …] eso significa que en la columna 0 la reina esta en la fila 2; en la columna 1 esta en la fila 5, etc.

3. Propuesta del código

- ✓ Generar X la cual es la solución inicial aleatoria con 8 reinas
- ✓ Calcular las colisiones que tendrá X
- ✓ Declaramos X* que vendría siendo la solución general quedaría “x*=x”
- ✓ Mientras que la solución general sea mayor que 0 y no se haya alcanzado su número de movimientos entonces:

a) Se genera N(X) son todos los vecinos los cuales vendrían siendo los tableros posibles moviendo solo una reina.
b) Elegimos X_prima es el vecino con menos colisiones que no estén en la lista tabú.
c) Movemos el nuevo estado y quedaría que X = X_prima.
d) Declaramos X_prima como el tabú y esto significa añadirlo a la lista y borrar el tabu más viejo si se llena.
e) Declaramos la siguente condición: SI f(X_prima) < f(X*) : -X* = X_prima esta condición servirá para actualizar nuestro mejor tablero histórico.
f) Aumentar el contador de movimientos

- ✓ Fin
- ✓ X* es el óptimo local solo sus choques son iguales a 0

Para esta propuesta se eligió esta propuesta porque de esta manera cumple cada condición y pasos del método tabú, a diferencia de otros métodos que construyen el tablero pieza por pieza en este caso vendría siendo X la búsqueda tabú necesita analizar todo el problema desde el principio a pasar de los errores, en el casos de

los vecinos estos solo se generaran dependiendo de la solución inicial además de que necesitamos saber cuales son las opciones inmediatas, porque elegimos el que no esta en la lista tabú de manera lógica es porque elegimos el mejor disponible el cual es el x' pero se puso de manera estricta en un filtro para la lista para la memoria del programa, en la regla tabú la cual es esta f(X_prima) < f(X*) nos obliga a hacer movimientos malos para aceptar tableros con mas colisiones temporalmente para salir de un ciclo, si no se guarda una copia del mejor tablero que vimos en X* se pierde para siempre y al final lo que se imprime es X*, y quisiera aclarar sobre los criterios de parada eso solo convertimos en un MIENTRAS o un while y porque lo escogimos de ese modo es porque no sabemos cuantos pasas tomara.

4. Código realizado en Python

```
Import random

Import time

N=8

Max_movimientos= 1000 #criterio de parada

Tamaño_lista_tabu = 15 #memoria del algoritmo

def contar_colisiones(tablero):

colisiones=0

for i in range (N)

for j in range(i+1,N)

#Regla 1

if tablero[i] == tablero[j]

colisiones = colisiones +1

#Regla 2

distancia_columnas = j-i

distancia_filas = abs(tablero[i] – tablero[j])

if distancia_columnas == distancia_filas:

colisiones = colisiones + 1

return colisiones

#Generador de solución inicial
```

```
estado_actual = []
for i in range(N):
estado_actual.append(random.randint(0,7))
#Guardar la mejor solución
mejor_estado_global = list(estado_actual)
mejor_colisiones_global = contar_colisiones(mejor_estado_global)
lista_tabu=[]
movimientos=0
tiempo_inicio = time.time()

#Ciclo Principal
While mejor_colisiones_global > 0 and movimientos < max_movimientos:
mejor vecino = []
colisiones_del_mejor_vecino = 999999 #para poder empezar se escoge un numero grande
#explora todos los movimientos posibles
for columna in range(N):
for fila in range(N):
#solo se evalua si es un movimiento diferente al que tuvo o tiene la reina
if fila != estado _actual[columna]:
vecino_simulado = list(estado_actual)
vecino_simulado[columna] = fila
#identificador si el tablero que analiza o realiza esta dentro de la lista tabu
es_tabu = false
for tablero_tabu in lista_tabu:
if vecino_simulado == tablero_tabu:
es_tabu = True
```

```
#Elegir el mejor vecino no tabu
If es_tabu == false
colisiones_simulados = contar_colisiones(vecinos_simulado)
if colisiones_simulados < colisiones_del_mejor_vecino:
colisiones_del_mejor_vecino = colisiones_simulados
mejor_vecino = list(vecino_simulado)
#mejor vecino encontrado
Estado_actual = list(mejor_vecino)
# declaramos x' como tabu
lista_tabu.append(list(estado_actual))
if len(lista_tabu) > tamaño_lista_tabu:
lista_tabu.pop(0)

if colisiones_ del_mejor_vecino < mejor_colisiones_global:
mejor_colisiones_global = colisiones_del_mejor_vecino
mejor_estado_global = list(estado_actual)
#fin del algoritmo
tiempo_fin = time.time()
tiempo_total = tiempo_fin – tiempo_inicio

print("=== resultado de la búsqueda tabu ===")
print("solución encontrada:",mejor_estado_global)
print("colisiones finales:" mejor_colisiones_global)
print("movimientos realizados: ", movimientos)
print("tiempo de calculo:", round(tiempo_total,4), "segundos")
```

Ahora quisiera dar una explicación de cómo es la lógica de este código, imaginemos que el algoritmo que tenemos es un explorador ciego en una montaña llena de valles y los valles son los tableros con pocas colisiones, en cada iteración el explorador investiga el terreno a su alrededor como el for que genera los vecindarios y busca el paso que los lleve hacia mas abajo o en el código el que tenga menos colisiones pero a veces el explorador encuentra un valle falso es cual en el código es un optimo

local donde todos los pasos a su alrededor son hacia arriba, en realidad si esto fuera un algoritmo tradicional esto se quedaría atrapado para siempre o estancado, pero gracias a la lista tabú nuestro algoritmo logra bloquear el fondo del valle en el código esto seria un criterio de parada para obligarlo a dar unos pasos hacia arriba temporalmente esto significaría aceptar un tablero con mas colisiones esto con el objetivo de escapar de ese hoyo y seguir buscando el valle real el cual se atribuye como encontrar el óptimo local el que tenga 0 colisiones en otra parte del terreno.

https://github.com/LeyCarlos-21170368/Topicos-de-IA.git